

Image

CHAPTER

1

Web Application Security Basics

The world depends on the Web. Web applications have, in a very short amount of time, become our primary means of shopping, banking, keeping up with the news, and ultimately, communicating. Web-based technologies have taken over the world as the primary way of building software applications. Even mobile applications, often touted as a new trend in software development, are basically web service–based thin client applications that depend on web technology to work effectively.

In the enterprise, the many benefits of the Java ecosystem have led the vast majority of companies to build their web platforms and web applications on Servlet and Java EE technologies. Like any programming language or framework, programmers using Java do not automatically gain the benefits of secure software. Developers need to understand how to use Java and a host of third-party libraries in specialized ways in order to build and deploy secure software. This book endeavors to help developers understand the theories behind secure software as well as provide guidance on how to use certain key techniques to defend their applications.

Thank you for starting this journey with us. As we named this book *Iron-Clad Java*, we envision this book to be the beginning of a series. We want to move every developer in the direction of Steel-Clad Java and Adamantium-Clad Java, and Self-Defending Laser-Powered-Armor-Clad Java, but our first honest step is Iron-Clad. The path to secure software is not an easy one and requires discipline, study, and a great deal of practice. We hope this book will guide you down this path in a way that benefits you, your team, and especially your users in positive ways.

And by the way, if you are a web application developer, we fully expect this book to be a positive financial investment and make you money. It’s difficult for companies to find developers who are knowledgeable in secure coding and secure software development. Those who truly understand security can command better positions and higher rates. If you read this book three times and truly master these techniques, we guarantee you will make more money as a software engineering professional.

What Is Untrusted Data?

While reading this book, you will frequently see the term “untrusted data.” But what does that mean? Untrusted data, at first glance, is any data that enters your system. This is data that is not hard-coded or produced by your application. You could think of untrusted data as raw input from your users. That is a good start, but that definition is unfortunately incomplete. Any data that enters your web application from an outside source, be it input typed in by users, uploaded files, third-party cloud services, even databases and other applications within your own organization, should be treated as untrusted.

To understand the potentially evil power of untrusted data, let’s first talk about the idea of an *intercepting web proxy*, a rather basic and fundamental tool that hackers will use as they attempt to exploit your web application. As you will see, even input that your users do not type in themselves should still be considered “untrusted data.”

Let's start by looking at an important network setting common to all browsers—your proxy server setting. In Firefox, for example, security testers (and hackers) will start by changing their network proxy setting to proxy all requests through some port on localhost (see [Figure 1-1](#)).¹

Image

FIGURE 1-1. *Changing proxy server settings in Firefox*

Once this setting is applied, the browser will stop working because a proxy server is expected at that address (see [Figure 1-2](#))! This is proper browser behavior until an appropriate proxy server is launched locally.

Image

FIGURE 1-2. *Browser is configured to connect to a nonexistent proxy*

Once you launch an intercepting proxy (such as Burp Suite, OWASP ZAP, OWASP WebScarab, and the Firefox plug-in Tamper Data), your browser once again starts surfing the Web. However, your traffic is now being recorded by the proxy (see [Figure 1-3](#)).

Image

FIGURE 1-3. *Web traffic being intercepted and recorded by OWASP ZAP*

Once the proxy is in place, individual requests can be intercepted and modified. Not only can attackers modify fields that contain user-entered input, they can also add, modify, or delete headers, cookies, the HTTP verb (GET, POST, and so on), or any other aspect of the HTTP request (see [Figure 1-4](#)). This leads us to the most fundamental rule of secure coding:

Image

FIGURE 1-4. *Editing an intercepted request with OWASP ZAP*

Â Â Â Â Â Â Â Image

Â Â Â Â Â Â Â **CAUTION**

Â Â Â Â Â Â Â *Untrusted data can be any part of the HTTP request! Do not trust any aspect of an HTTP or HTTPS request! Attackers can and will modify any aspect of the request in order to exploit your web application.*

What about SSL? HTTPS is encrypted so it is safe, right? In fact, HTTPS only protects data in transit and usually only the server endpoint is authenticated. Client-side certificates are unusual. Even if the client is authenticated using a client-side certificate, his or her computer could still be compromised by malware or by an attacker simply sitting down at an unlocked workstation. The bottom line is that you can't trust data received from the client, even from trusted users.

Is the HTTP request the only untrusted data? Security should be easy then—just validate every input and you should never have a breach, right? Unfortunately, modern web applications don't just rely on input from the user surfing the site. Applications also use data from databases, RSS feeds, web services, data created by other users or administrators of the system, and a whole host of other sources. To truly protect your web applications, you must also treat all of these other sources as untrusted data and handle them safely as well. As you will see later in this book, safe data handling means properly validating all sources of data that enter your web application and also properly encoding data when it exits the application to a web page, database, or other location.

We think of this situation as *security quality*. When we write programs, we include business rules to ensure that data is consistent with business requirements. Security is really much the same with one noteworthy exception: Most security requirements are not explicitly defined. Instead, they are most often lumped into nonfunctional business requirements like performance. In some sense, security requirements have to be fluid in order to adapt to the ever-evolving threat landscape. However, as you will see in this book, there are actually quite a few known threats that we as web developers *must* plan for in order to build successful, secure applications.

HTTP Security Considerations

It's important to have a fundamental understanding of the HTTP protocol in order to build a secure web application. HTTP, or Hypertext Transfer Protocol, is a simple, stateless, request and response protocol that drives the vast majority of web traffic.

HTTPS

HTTPS, or Hypertext Transport Protocol Secure, provides a variety of different encryption services to protect user data during transport, such as protecting data while it's traveling over a wired or wireless network. HTTPS allows for confidentiality (to protect your data from being seen), integrity (to protect your data from being changed), and authenticity (to assure you that the domain you are visiting is really the real version of that domain), among other benefits.

Not all HTTPS is created equal. Cipher suites are negotiated between web browser and server to determine the cryptographic strength of the session transport. The negotiation process ensures that different types and versions of web browsers and web servers collaborate over unsecured networks securely. This means that the cryptographic strength of an HTTPS connection can vary widely between different HTTPS servers. Most browsers will display a lock icon or some other indication that HTTPS is in use, but ultimately the negotiated cipher suite results are not obvious to users when browsing websites. We discuss HTTPS in more depth in [Chapter 6](#), “Protecting Sensitive Data.”

HTTP/S GET Request

The HTTP request consists of several basic components. These include the HTTP verb, the resource requested, the version of HTTP, the headers, and parameters.

Image

In the preceding example, the HTTP verb is `GET`, the resource being requested is `http://www.oracle.com/index.html` and the version is `HTTP/1.1`. The HTTP request headers include `User-Agent`, `Accept`, `Accept-Language`, `Connection`, and `Host`. In a GET request, the parameters are appended to the URL after a question mark and separated by an ampersand (&). The parameters in this request are `locale`, with the value `US`, and `lang` with the value `en`.

The User-Agent HTTP request header identifies the type of client making the request. It can be a desktop browser, a mobile browser, a web crawling bot, a scanner, or a mobile application. In the preceding case, the User-Agent identifies itself as Mozilla/5.0 (Macintosh; Intel Mac OS X 10.9; rv:29.0) Gecko/20100101 Firefox/29.0, which is Firefox 29 on my OS X machine. Because this header comes from the browser and is *not* typically typed in by a user, can you trust it? Of course not. An attacker can use an intercepting proxy or similar tool to easily modify any part of the request to something malicious.

Image

Many user actions will trigger an HTTP request from the browser. Clicking on a link, typing a URL in the URL bar, submitting a form, selecting a bookmark, or even simply keeping a web page open in the browser may trigger AJAX requests. Unfortunately, HTTP (and even HTTPS) GET requests leak like a sieve.

For example, let's take a look at [Figure 1-5](#), which is my browser history from today. Not only was this HTTP request to manico.net leaking data over HTTP (which is unencrypted), but it is also leaked in the browser history of Firefox and other browsers.

Image

FIGURE 1-5. *Firefox browser history showing leaked GET parameters*

Even when a site is requested over HTTPS, the full GET request is still visible in the browser history, as shown in [Figure 1-6](#).

Image

FIGURE 1-6. *Even HTTPS GET requests can leak sensitive information in the browser history.*

HTTP GET requests also leak over HTTP referrer headers. HTTP referrer headers are HTTP request headers that are added by browsers to provide websites with the URL that the request originated from. For example, if you click a link on website A that takes you to website B, then the HTTP referrer header in the request to website B would include the full URL of website A.

For example, the following request to a cruise website includes a referer header from an advertising network (googleads.g.doubleclick.net). This is valuable to the cruise company because it allows them to track which advertising network the request came from.

Image

But if sensitive data is included in the URL, the referer header will leak that to other sites:

Image

HTTP/S POST Request

The HTTP POST request is a more secure way to transport sensitive data. The following POST has three properties that allow for the secure transport of sensitive data: The request is an HTTPS request, the HTTPS verb is POST, and the sensitive data is in the body of the request instead of the URL.

Image

By putting the parameters in the body of the request instead of the URL, the submitted data is not leaked in the browser history list, referer headers, or server access logs.

^ ^ ^ ^ ^ ^ ^ Image

^ ^ ^ ^ ^ ^ ^ **TIP**

^ ^ ^ ^ ^ ^ ^ *Submit sensitive data only over HTTPS POST in the POST body!*

HTTP/S Response

When an HTTP server receives a request, it will return an HTTP response, which includes the HTTP version and response code, headers, and the response body. The response body is typically HTML, but it can also be XML, JSON, or a binary file such as an image or document. A typical HTTP response will look similar to the following:

Image

The first line of the preceding response indicates the request was successful (200 OK). The response also includes headers that indicate how the browser should handle the response (Content-Type, Strict-Transport-Security, and so on). The Set-Cookie header tells the browser to send a JSESSIONID cookie containing a unique sequence of numbers and letters with each following request. This will allow the server to maintain a session and personalize content for the user. Finally, the body of the response contains the actual HTML page that will be rendered by the browser.

HTTP/S Response Headers

In recent years, a variety of security-centric HTTP response headers have become standardized across the majority of browsers. These “browser standard” response headers are often easy ways to add security to your web application.

Cache-Control HTTP Response Header

One of the older HTTP security response headers is the Cache-control HTTP response header. This response header is a directive to tell the browser not to cache sensitive data.

Image

No-cache says that the browser should revalidate the data with every request. No-store says that no part of the request or response should be stored at all in the first place. Must-revalidate says that under no circumstances should stale content be served by the browser.

The Expires header tells the browser when the content should expire and thus be requested again from the server. A negative value or a value in the distant past should always force content to immediately expire.

Note that these headers can be extremely useful for improving the performance of your website by caching static content such as images and stylesheets on users’ browsers for long periods of time. This will lessen bandwidth consumption and make the server more resistant to denial-of-service attacks. Care must be taken to ensure that such static content is cached, but any sensitive or user-specific data is never stored.

Strict Transport Security HTTPS Response Header

The `Strict-Transport-Security` response header (HSTS response header) is a simple way to force your browser to always use the HTTPS protocol for a certain amount of time. This header must be delivered over an HTTPS connection for it to take effect. The following response header, when delivered over HTTPS, will force supported browsers to only use HTTPS for the next 3153600 seconds when contacting that site, including all subdomains:

Image

HSTS was deemed a standard by the IETF (RFC 6797) in late 2012. Browser support is mixed. While Firefox and Chrome have supported HSTS headers since version 4 of both browsers, Internet Explorer does not support HSTS as of March 2014 but plans to support it in the future when IE 12 is published.

X-Frame-Options HTTP Response Header

Clickjacking is a form of attack that tricks a user into thinking he is clicking on one location when he is really clicking on another location that could cause harm. One of the main defenses used to stop clickjacking is the `X-Frame-Options` response header. We discuss this risk in detail in [Chapter 5](#).

X-XSS-Protection HTTP Response Header

Internet Explorer 8 and above support the `X-XSS-Protection` header to help stop some forms of cross-site scripting.² This is enabled by default for most IE security configuration settings, but can be forcibly disabled with the `X-XSS-Protection: 0` response header. This filter can also be forcibly enabled in “block” mode with the header `X-XSS-Protection: 1; mode=block`. While this filter is far from perfect (it is only supported by IE and only stops some forms of XSS), enabling it as a good defense-in-depth mechanism is prudent. We discuss XSS defense, in depth, in [Chapter 4](#).

While this list of response headers is not all-inclusive, these are some of the most important response headers that developers need to master in order to build a secure web application.

Anti-Patterns and Weaknesses

Throughout this book, we will describe a variety of vulnerabilities that you may incidentally create in your application. First of all, there is no shame in authoring a security vulnerability in your code. Developers are often at an extreme disadvantage when it comes to secure software development. How many developers learned about secure coding techniques in school? Even to this day, very few university computer science programs teach the art and science of secure software design and development. Although Java is a language that we cherish, it’s a tool that must be used properly in order for our applications to be secure. The frameworks that we depend upon also need to be mastered and used in a secure fashion. So while there is no shame in creating vulnerabilities in code, we need to master the proper coding techniques to avoid them. Here are some of the most common anti-patterns

Blacklist Input Validation

One of the primary defense techniques that we will discuss is input validation. Input validation should seek to define what constitutes good data and reject everything else (whitelisting). When validation layers seek to only block input that is known to be dangerous (blacklisting), the

defense layer is weak and can be evaded. For example, check out the OWASP Filter Evasion Cheatsheet,³ which provides a wide variety of attacks to evade cross-site scripting blacklist filters! More on cross-site scripting in [Chapter 4](#).

Lack of Parameterized SQL

It's all about the database! Web applications are often glorified database applications. Developers use a significant amount of SQL in their application code. When developers build SQL without using parameterized queries, it can lead to the quite dangerous vulnerability, SQL injection. We discuss SQL injection and other forms of injection in [Chapter 7](#).

Use of Weak or Incorrect Ciphers

When handling sensitive data, you need to encrypt that data both in transit and at rest. Developers need to understand which cryptographic algorithm to use and which cipher modes are most prudent, at least! We discuss applied crypto at rest and in transit in [Chapter 6](#). Cryptography for proper password storage is a specialized topic and is discussed in [Chapter 2](#).

This is only the beginning, but these anti-patterns should give you an idea as to the type of software coding problems (and solutions!) that we will explore.

Security Controls and Positive Patterns

So far in this chapter, we have come up with two very important secure coding rules: (1) do not trust anything from the request and (2) only submit sensitive data over HTTPS POST in the body of the POST. But this is just the beginning. As we dive deeper into the many aspects of secure software, we will discuss a wide variety of positive defensive patterns such as those found in the next two sections.

Verify Authentication and Authorization with Every Request

Some frameworks or applications require that programmers manually add checks to see if a user is logged in with every individual action. Some use filters to ensure that all requests must be authenticated. Regardless, it is critical to ensure that all authenticated actions do indeed contain a proper and active session. We discuss authentication and session management in detail in [Chapter 2](#).

In addition to authentication, it is also critical to ensure that the user is properly authorized for all actions (that is, the user has the right access control permissions). We discuss access control design in [Chapter 3](#).

Protect Transactions with the Synchronizer Token Pattern

One of the most dangerous attacks against a web application is Cross Site Request Forgery. All sensitive transactions must be protected with a synchronizer token or similar. We discuss Cross Site Request Forgery and various CSRF defense strategies in [Chapter 5](#).

This is only the beginning. We will discuss a wide variety of positive security patterns in the chapters to come.

Input Validation

Adversaries often gain a foothold into software applications by sending cleverly constructed inputs to negatively influence a program's behaviors. Input that is inconsistent with expected functionality and design will often cause the application to act in unexpected ways. Garbage in, pure exploitation gold out. This section discusses one of the most fundamental secure coding techniques: input validation. Input validation is one of the "first lines of defense" when in any web application.

Adversaries hack your web applications by adding attack strings to legal input fields, cookies, and other parts of the HTTP request. Very often this is done via modifications that still leave the HTTP request as a legal request: The HTTP protocol itself is not made invalid by these attacks. Input validation is a programming technique that limits what input a web application will legally accept. Proper input validation defines what data is allowed and rejects the rest. After passing validation, data can then be processed by your web application's business logic layer. This is one of several important layers of defense needed in a secure web application.

One of the primary technical artifacts that we use for input validation is a *regular expression*. Regular expressions consist of commands that contain sequences of metacharacters. These commands are used in string matching operations and are the most commonly used input validation technique.

Input Validation Anti-Patterns: Blacklist Validation Only

Blacklist or negative validation defines what an attack looks like and blocks it. This is an interesting and sometimes effective form of intrusion detection, but it is not a strong defense. Blacklist validation may stop some very dangerous well-known attacks to your application, but it is not a control that will stop all attacks. There is a huge incentive and relatively little risk for hackers to try new things. This means that the threat landscape will be constantly evolving and there are essentially an infinite number of possible attack patterns to block. There are many types of "filter evasion techniques" such as encoding your attacks or submitting legal input that might contain an attack (like the SQL injection email address ' - -@manico.net). Commercial "fuzzers" exist that can constantly bombard your applications with tiny variations of inputs until a weakness is found. These alone can make maintaining a blacklist a daunting chore, at best. In [Chapter 9](#), we will discuss means of using blacklists to detect *known* attacks, but blacklisting is by no means a defense against *all* attacks. So while blacklist validation could be one piece of the puzzle for securing your application, do not depend on it.

Input Validation Positive Patterns: Whitelisting

Rather than starting by attempting to block characters that are known to be bad (blacklisting), we should instead begin by defining only those characters and patterns that are known to be good (whitelisting). For example, a phone number input field should only need to accept digits. Even if we allow users to submit formatted phone numbers (for example "+1 (518) 555-5785"), a very limited range of characters are acceptable. These include digits, spaces, hyphens, parentheses, and possibly the plus sign for international numbers. Input that contains anything except this narrow range of acceptable characters and patterns (the whitelist) should be rejected and logged as a potential attack. Sometimes bad input is a user mistake, sometimes it falls in a gray area, and sometimes it is clearly an attack. By logging these input validation errors, we provide our operations teams with an opportunity to review potential security issues.

Image

NOTE

Deny by default. Reject all characters except for specific characters of interest.

Input Validation Anti-Patterns: Regular Expressions and ReDoS

Regular expressions are frequently used to ensure that input conforms to some pattern, such as an email address being a sequence of letters and numbers, followed by an @ sign, followed by another sequence of letters, numbers, and periods. A regular expression denial-of-service vulnerability (ReDoS) occurs when regular expressions are authored in such a way that the time it takes to compute the regular expression grows exponentially related to input size. Attackers can exploit such a vulnerability to cause a denial of service in your application by sending a relatively tiny amount of data and forcing your application to consume a huge number of server cycles in validating it. A good tool for testing Java regular expressions is SafeRegex, by Sebastian [KÃ¼beck](#).⁴

Input Validation: Apache Struts

Almost every Java framework includes some form of input validation layer. In order for it to work, you need to use it. For example, Apache Struts 2.3 provides an extensible and feature-rich input validation framework, which includes features such as internationalization within error messages and more. Here is an example of a basic date range input validation configuration segment.

Image

Apache Struts 2.3 also provides a variety of default validators beyond simple regular expressions. These include:

Image

For more information on Apache Struts 2.3 input validation, please visit <http://struts.apache.org/release/2.3.x/docs/validation.html>.

Basic Input Validation Considerations: Length of Input

One often overlooked area of input validation is to validate the minimum and maximum length of user input. Validating the length of user input is a simple task either through string APIs or via regular expressions. If you forgot this step, users can send incredibly large amounts of user input into your application and cause harm. The Django framework experienced a major denial-of-service vulnerability because it did not validate the length of user input for passwords.⁵ What can you expect from a project that uses an inferior language named after a snake? Weâ€™re coffee people here!

Validating Numerical Input

One of the beautiful aspects of the Java language is the various numerical classes. Validating raw user input that should be a numeric class is a rather simple endeavor. All numeric classes contain a static parse function that takes raw string input and converts it to a formal numeric class. For example:

Image

will take a string and convert it to an instance of `Integer` and throw a `NumberFormatException` if the input is not an integer. These classes are perfect for

numerical input validation. As soon as possible, get your usersâ€™ string numerical input into a formal Java numeric class!

Validating Open Text Input

Most news websites, social media, blogs, and similar sites allow users to submit comments as open text. What characters are you going to allow? Uppercase and lowercase letters? Numbers? Punctuation? Guess whatâ€”that is all an attacker needs to cause harm to your application. Validating open text does indeed reduce the attack surface of your application (in other words, less input will be accepted than if input validation did not exist in your application). But validating open text input does not necessarily make that input secure. Many secure coding books praise input validation as the primary defense layer for software security. We feel that query parameterization, encoding, and escaping are all much more important than input validation as a defense against malformed inputs. Input validation is indeed important as part of a strong defense in depth, but do not depend on input validation alone!

^ ^ ^ ^ ^ ^ ^ Image

^ ^ ^ ^ ^ ^ ^ **TIP**

^ ^ ^ ^ ^ ^ ^ *Apply input validation routines to all input but do not depend on it, especially if that input is open text.*

Input Validation Positive Patterns: URL Validation

Validating a URL from a user (such a link from social media) is a challenging task. Using a regular expression to validate a URL can be a complex and error-ridden affair. The following code snippet from Jeff Ichnowski (the author of the OWASP Java Encoder Project) demonstrates the complexity of validating an untrusted URL.

Image

Where Do We Go from Here?

Jeremy Long, a senior application security expert, once told me, â€œI can build a completely secure web application and skip all input validation.â€ I applauded the depth of his comment. While input validation is an important defensive layer to limit what input a user may submit into an application, itâ€™s often not a layer you can depend on, especially when validating complex string input. As this book unfolds, we will describe, in depth, a variety of other measures to help defend your application even when validated user input still contains attack data. Letâ€™s dig in!

¹ A useful plug-in for Firefox, IE, and Chrome is FoxyProxy. It allows you to set up multiple proxies and route requests through different proxies based on the URL. This is helpful so that you are using only your intercepting proxy for the site you actually want to test, and not proxying all of your web traffic.

² <http://blogs.msdn.com/b/ieinternals/archive/2011/01/31/controlling-the-internet-explorer-xss-filter-with-the-x-xss-protection-http-header.aspx>

³ https://www.owasp.org/index.php/XSS_Filter_Evasion_Cheat_Sheet

⁴ <https://code.google.com/p/saferegex/>

⁵ <http://arstechnica.com/security/2013/09/long-passwords-are-good-but-too-much-length-can-be-bad-for-security/>